Database Development and Design (SOE201)

Lecture 5a:
Query Evaluation Part 1

Dr. Wei Wang

AIAT

Learning Outcomes

- Algorithms for Evaluating Selection
- Algorithms for Evaluating Projection
- Algorithms for External Sorting

Selection Operator

- Notation: $\sigma_p(r)$
- p is selection **predicate**
- Defined by:

  $$\sigma_p(r) = \{t \mid t \in r \text{ and } p(t)\}$$

  in which p is a formula of propositional calculus of terms connected by: $\wedge$ (**and**), $\vee$ (**or**), $\neg$ (**not**)
  Each term is of form:

  <attribute> *op* [<attribute> or <constant>]

  where *op* can be one of: $=, \neq, >, \geq, <, \leq$
- Selection example:

  $\sigma_{branch\text{-}name='Perryridge'}(account)$

Evaluation of Selection

- File scan – search algorithms that scan files and retrieve records that fulfill selection condition.
  - Blocks of relation are stored contiguously
  - Also called linear scan
- Index scan – search algorithms that use index
  - Selection condition must be on search key of index
    - By default, use B^+-tree index

Algorithm for Selection (File Scan: *Linear Search*)

- Scan each block and test all records to see whether they satisfy selection condition.
  - Cost estimate = b_r block transfers + 1 seek
    - b_r denotes number of blocks containing records from relation r
  - If selection is on key attribute, can stop on finding record (Linear Search, Equality on Key)
    - Average cost = (b_r/2) block transfers + 1 seek
  - Linear search can be always applied, regardless of:
    - Selection condition
    - Ordering of records in file, or
    - Availability of indices

Algorithms for Selection (File Scan: *Binary Search*)

- Applicable only if selection is equality comparison on attribute on which file is ordered.
  - Cost estimate (number of block transfers and seeks):
    - Cost of locating first tuple by binary search on blocks
      - $\lceil \log_2(b_r) \rceil$ block transfers and $\lceil \log_2(b_r) \rceil$ seeks
      - In time: $\lceil \log_2(b_r) \rceil * (t_T + t_S)$
    - If there are multiple records satisfying selection
      - Add transfer cost of number of blocks containing records that satisfy selection condition
  - If b_r is not too big, then most likely binary search doesn't pay.
    - Note that t_T is several (say, 40) times smaller than t_S
    - Two algorithms needs to be wisely chosen for specific query at hands.

Selection Using Indices (*Primary Index on Candidate Key, Equality*)

- Cost to retrieve single record that satisfies corresponding equality condition
  - $(h_i + 1)$ block transfers, and $(h_i + 1)$ seeks
    - h_i is height of B+-tree index
  - In time: $(h_i + 1) * (t_T + t_S)$
- Recall that height of B$^+$-tree index is at most $\lceil \log_{\lceil n/2 \rceil}(K) \rceil$, where n is number of pointers per node and K is number of unique search keys.
  - E.g. for relation with 1,000,000 (1 million) different search keys, and with 100 index entries per node, $h_i = 4$
  - Unless relation is really small, this algorithms always "pays" when indexes are available.

Selection Using Indices (*Primary Index on Non-key, Equality*)

- Retrieve possibly multiple records.
  - Records are on consecutive blocks
  - Cost:
    - *(h_i + b) block transfers and (h_i +1) seeks*
      - *b* is number of blocks containing matching records
      - Once first record in found, rest can be read sequentially
    - *In time*
      - (h_i + b) * t_T + (h_i +1) * t_S , or
      - h_i *(t_T + t_S) + t_S + t_T* b

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Selection Using Indices (*Equality on Candidate Key, Secondary Index)*

- Retrieve single record as search-key is candidate key
  - (h_i + 1) block transfers and (h_i + 1) seeks
  - *In time:* (h_i + 1) * (t_T + t_S)

Selection Using Indices (*Equality on Non-key, Secondary Index)*

- Retrieve multiple records if search-key is not candidate key
  - Each of *n* matching records may reside on different block
  - Cost:
    - *(h_i + n) block transfers and (h_i + n) seeks*
    - Can be very expensive if *n* is big.
      - Note *n* multiplies time for seek.

Comparative Selection $\sigma_{A \leq V}(r)$ or $\sigma_{A \geq V}(r)$

- Using linear scan or binary search just as before.
- Using primary index, comparison
  - For $\sigma_{A \geq V}(r)$, use index to find first tuple $\geq v$ and scan relation sequentially from there
  - For $\sigma_{A \leq V}(r)$ just scan relation sequentially till first tuple $> v$;
    - Using index would be useless, and require extra seeks on index file.
- Using secondary index, comparison
  - For $\sigma_{A \geq V}(r)$ use index to find first index entry $\geq v$ and scan index sequentially from there, to find pointers to records.
  - For $\sigma_{A \leq V}(r)$ just scan leaf pages of index to find pointers to records, till first entry $> v$
  - In either case, retrieving records
    - Requires I/O for each record (lot)
    - Linear file scan may be much cheaper

Conjunctive Selection $\sigma_{\theta 1 \wedge \theta 2 \wedge \cdots \wedge \theta n}(r)$

- *Using one index*
  - First select θ_i and previous algorithms that results in least cost for $\sigma_{\theta i}(r)$.
  - Then test other conditions on tuples after fetching them into memory buffer.
  - Choice of first condition is crucial.
    - One must use estimates to know which one is best.
- *Using multiple-key index*
  - Use appropriate composite (multiple-key) index if available.

Disjunctive Selection $\sigma_{\theta 1 \vee \theta 2 \vee \cdots \vee \theta n}(r)$

- Use linear scan.
- Or, use index scan if *some* conditions have available indices.
  - Use corresponding index for each condition, and take union of all obtained sets of record pointers.
  - Then fetch records from file

Selection With Negation: $\sigma_{\neg\theta}(r)$

- Use linear scan
- If index is applicable to θ
  - Find satisfying records using index
  - $\sigma_{\neg\theta}(r)$ is simply set of tuples in r that are not in $\sigma_{\theta}(r)$

Evaluating Projection

- Projection drops columns not in selected attribute list.
- Expensive part is removing duplicates.
- Duplicate elimination can be implemented via hashing or sorting.
  - On sorting, duplicates will come adjacent to each other, duplicates can be deleted.
  - Hashing is similar; duplicates will go into same bucket.

Sorting

- Sorting algorithms are important in query processing at least for two reasons:
  - Query itself may require sorting (***order by*** clause)
  - Some algorithms for other operations, like merge join, set operations and aggregation, require that relations are previously sorted and duplicates removed.
- To sort relation:
  - May build index on relation, and then use index to read relation in sorted order.
    - This only sorts relation *logically*, not *physically*
    - May lead to one disk block access for each tuple.
  - For relations that *fit in memory*, sorting algorithms like quicksort, selection and insertion sort, can be used.

External Sort-Merge

- For relations that *don't fit in memory*, need special algorithms, e.g. external sort-merge.
  - Take into account block transfers and seeks.
- Let *M* denote memory size: number of disk blocks whose contents can be buffered in main memory.
- First thing to do is to ***create sorted runs***.

Let i be 0 initially.
Repeat following till end of relation:
(a) Read M blocks of relation into memory;
(b) Sort in-memory blocks;
(c) Write sorted data to run file R_i;
(d) Increment i.
Let final value of i be N (that is, there are N run files)

- *Next step: merge sorted runs*

External Sort-Merge cont'd

Merge sorted runs (N-way merge): assume $N < M$.

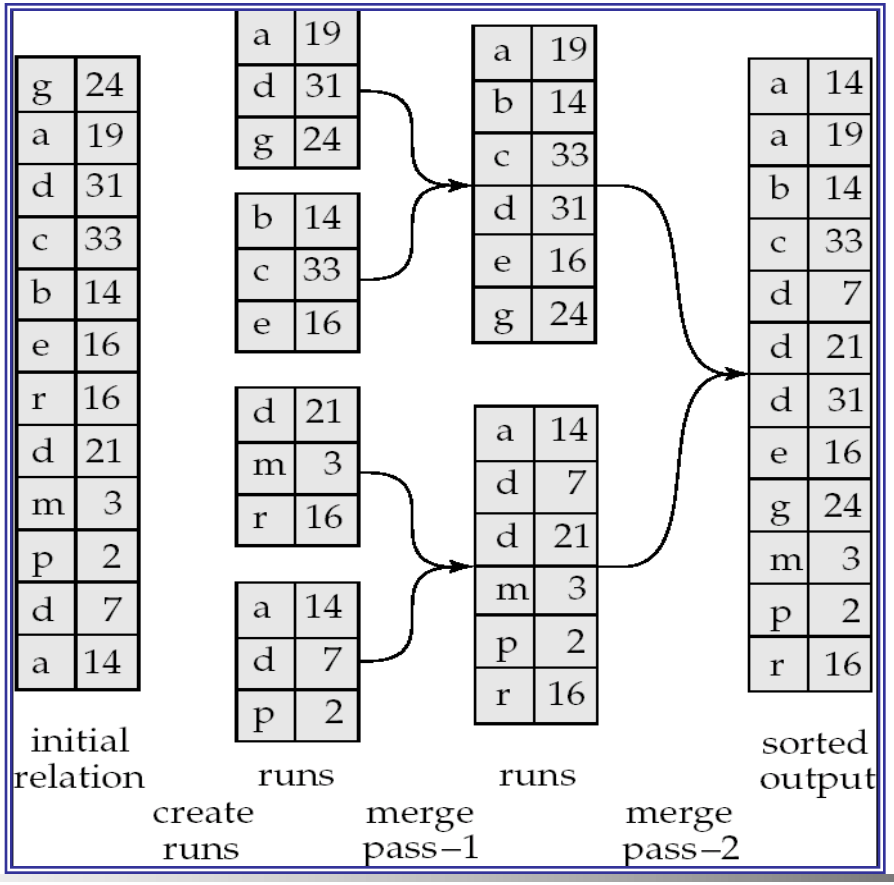
1. Use N blocks of memory to buffer input runs (1 for each of N run files), and 1 block to buffer output. Read first block of each run into memory.
2. **Repeat**
   1. Select first record in sort order (i.e. smallest) among all buffer pages
   2. Write record to output buffer. If output buffer is full write it to disk.
   3. Delete record from its input buffer page.
   4. If buffer page becomes empty then read next block (if any) of run into buffer.
3. **Until** all input buffer pages are empty

External Sort-Merge (Cont.)

- If $N \geq M$, several *merge passes* are required.
  - In each pass, contiguous groups of $M - 1$ runs are merged.
  - One pass reduces number of runs by factor of $M - 1$ (and creates runs longer by same factor).
    - For example, if $M=11$, and there are 90 runs, one pass reduces number of runs to 9, each 10 times size of initial runs
  - Repeated passes are performed till all runs have been merged into one.

Example: External Sorting Using Sort-Merge

Suppose that relation has 12 blocks; one tuple fits in one block; memory holds at most three blocks.



Cost Analysis

- Assume relation in b_r blocks, memory size M, number of run file $\lceil b_r/M \rceil$.
- Buffer size b_b (read b_b blocks from each run and b_b blocks for output writing; before it was assumed $b_b=1$).
- Cost of Block Transfer
  - Each time can merge $\lfloor (M-b_b)/b_b \rfloor$ runs, i.e. $\lfloor (M/b_b \rfloor -1$;
  - So, total number of merge passes required: $\lceil \log_{\lfloor M/bb \rfloor -1} \lceil b_r/M \rceil \rceil$.
  - Block transfers for initial run creation as well as in each pass is $2b_r$ (read/write all b_r blocks).
  - Thus total number of block transfers (*For final pass, we don't count write cost*):

    $2b_r + 2b_r \lceil \log_{\lfloor M/bb \rfloor -1} \lceil b_r/M \rceil \rceil - b_r$

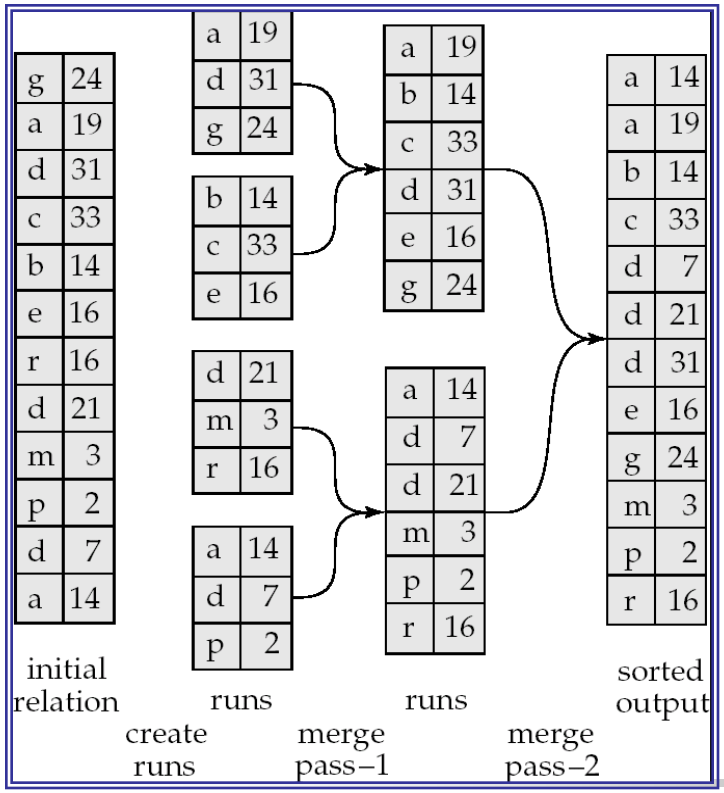
    $= b_r (2 \lceil \log_{\lfloor M/bb \rfloor -1} \lceil b_r/M \rceil \rceil + 1)$
- Cost of seeks
  - During run generation: one seek to read each run and one seek to write each run $2\lceil b_r / M \rceil$
  - During merge phase: need $2\lceil b_r / b_b \rceil$ seeks for each merge pass
  - Total number of seeks:

    $2\lceil b_r / M \rceil + 2\lceil b_r / b_b \rceil \lceil \log_{\lfloor M/bb \rfloor -1} \lceil b_r/M \rceil \rceil - \lceil b_r / b_b \rceil$

    $= 2\lceil b_r / M \rceil + \lceil b_r / b_b \rceil (2 \lceil \log_{\lfloor M/bb \rfloor -1} \lceil b_r/M \rceil \rceil - 1)$

Example: External Sorting Using Sort-Merge

Suppose that Memory hold at most 3 blocks, M=3. There are 12 blocks in relation and only one tuple fits in block. *Assume* b_b =1.



Total block transfer = ?

Total seeks = ?

End of Lecture

- Reading
  - Textbook Chapters 15.1, 15.2, 15.3, 15.4